Midterm Exam, Faculty of Engineering, Chulalongkorn University
Course ID: 2110215 Course Name: Programming Methodology I
Second Semester, Date: 6 March 2020 Time: 8.30-11.30AM

Name ..... Student ID. ..... No. in CR .....

Instructions

1. Write your Student ID, full name, and your number in CR58 in the space provided on this page.
2. Your answer must be on the computer center's machine in front of you.
3. Documents and files are allowed inside the exam room; however, internet and flash drive are prohibited. Borrowing is not allowed unless it is supervised by the proctor.
4. You must not carry mobile phone and flash drive during the exam.
5. ***** You must not bring any part of this exam paper outside. The exam paper is a government's property. Violators will be prosecuted under a criminal court and will receive an F in the subject. *****
6. Students who wish to leave the exam room before the end of the exam period, must raise their hands and ask for permission before leaving the room. Students must leave the room in the orderly manner.
7. Once the time expires, student must stop typing and must remain seated quietly until the proctors collect all the exam papers or given exam booklets. Only then, the students will be allowed to leave the room in an orderly manner.
8. Any student who does not obey the regulations listed above will receive punishment under the Faculty of Engineering Official Announcement on January 6, 2003 regarding the exam regulations.
    a. With implicit evidence or showing intention for cheating, student will receive an F in that subject and will receive an academic suspension for 1 semester.
    b. With explicit evidence for cheating, student will receive an F in that subject and will receive an academic suspension for 1 year.

Please sign and submit

Signature (.....)

Important Rules

- You must not bring any part of this exam paper outside. The exam paper is a government's property. **Violators will be prosecuted under a criminal court and will receive an F in the subject.**
- It is a student's responsibility to check the file. If it is corrupted or cannot be open, there is no score.
- For each question, a table is given, showing (color-coded) whether or not you have to modify or create each method or variable.

** Noted that Access Modifier Notations can be listed below*

+ (public)
# (protected)
- (private)
<u>Underline</u> (static)
Italic (abstract)

Set-Up Instruction

- Set workspace to "**C:\temp\progmeth2019_2\Midterm_2110215_(your id)_(FirstName)**" (if not exist, you must create it)
  - For example, "C:\temp\progmeth2019_2\Midterm_2110215_6170156021_Boss"
- All your files must be in the workspace.
- **The code outside of the workspace will not be collected, or graded**

Scoring (Total 30 points, will be scaled to 40 points)

- Part1 = 10 points
- Part2 = 10 points
- Part3 = 10 points

Part 1: OOP Concept

Objective

1) Be able to implement Objects and Classes.

Instruction

1) Create Java Project named "**2110215_Midterm_Part1**".
2) Copy all folders in "**toStudent/Part1**" to your project directory src folder.
3) You are to implement the following classes (detail for each class is given in section 3 and 4)

   a) **CPShop** (package application)
   b) **Item** (package logic)
   c) **OrderItem** (package logic)
   d) **Order** (package logic)
4) **JUnit for testing is in package test.grader**

Problem Statement: CP Shop

CP Shop is a JAVA application for the shop manager to add items and create orders. The order contains item orders that tell which item the order has and how much the amount is. However, the current system's implementation is still incomplete. You are tasked to implement the remaining Item, Order and OrderItem classes.

Here's is how the Shop should work, part of this is already provided.

When the application is open, there will be a welcome message with all the available commands for the user.

```
===========CP Shop===========
What do you want to do?
[1] Add New Item
[2] List All Items
[3] Create New Order
[4] List All Orders
[5] Quit
> Please select your option:
```

Figure 1. Welcome screen when the application starts

1) The first option is adding a new item to the system. Please note that the new item name has to be different from existed items and not blank. Otherwise, the system will raise an error. If price per piece entered is less than 1, the system will set the item price to 1.

```
> Please select your option:    1
========================================
> Please enter item name:
Sange
> Please enter price per pieces of item:
2050
Sange (2050) has been added to item list successfully!
========================================
```

Figure 2. Successfully adding a new item

2) The user can choose to list all current items in the system. Each item information displays the item name and its price per piece. When the program starts, there will already have 4 initial items in the system.

```
> Please select your option:    2
========================================
[0] Yggdrasil Leaf price per piece: 4000
[1] Red Potion price per piece: 100
[2] White Potion price per piece: 215
[3] Blink Dagger price per piece: 2250
========================================
```

Figure 3. Initial Item Listing

3) Third option is creating a new order. The user will then be asked to select item by enter its index and then enter the amount to add to the order. The user can finish creating order by pressing Enter key.

If the user select to add item that already in the order, the amount of that item in the order will be increased.

```
> Please select your option:    3
========================================
[0] Yggdrasil Leaf price per piece: 4000
[1] Red Potion price per piece: 100
[2] White Potion price per piece: 215
[3] Blink Dagger price per piece: 2250
> Enter index of item to add to order / Enter blank to end order.
0
> Please enter of amount of the item.
40
> Enter index of item to add to order / Enter blank to end order.
3
> Please enter of amount of the item.
1
> Enter index of item to add to order / Enter blank to end order.
0
> Please enter of amount of the item.
60
> Enter index of item to add to order / Enter blank to end order.

Order No.0 created successfully!
========================================

===========CP Shop===========
What do you want to do?
[1] Add New Item
[2] List All Items
[3] Create New Order
[4] List All Orders
[5] Quit
> Please select your option:    4
========================================
Order No.0 : total price 402250
 -- Yggdrasil Leaf 100 piece(s) * 4000 per piece.
 -- Blink Dagger 1 piece(s) * 2250 per piece.
========================================
```

Figure 4. Creating a new order.

4) The user can list out all order in the system.

```
> Please select your option:    4
==========================================
Order No.0 : total price 402250
 -- Yggdrasil Leaf 100 piece(s) * 4000 per piece.
 -- Blink Dagger 1 piece(s) * 2250 per piece.
Order No.1 : total price 43000
 -- White Potion 200 piece(s) * 215 per piece.
Order No.2 : total price 338500
 -- Yggdrasil Leaf 50 piece(s) * 4000 per piece.
 -- Red Potion 300 piece(s) * 100 per piece.
 -- White Potion 400 piece(s) * 215 per piece.
 -- Blink Dagger 10 piece(s) * 2250 per piece.
==========================================
```

Figure 5. Listing all orders.

Implementation Detail

The class package is summarized below.

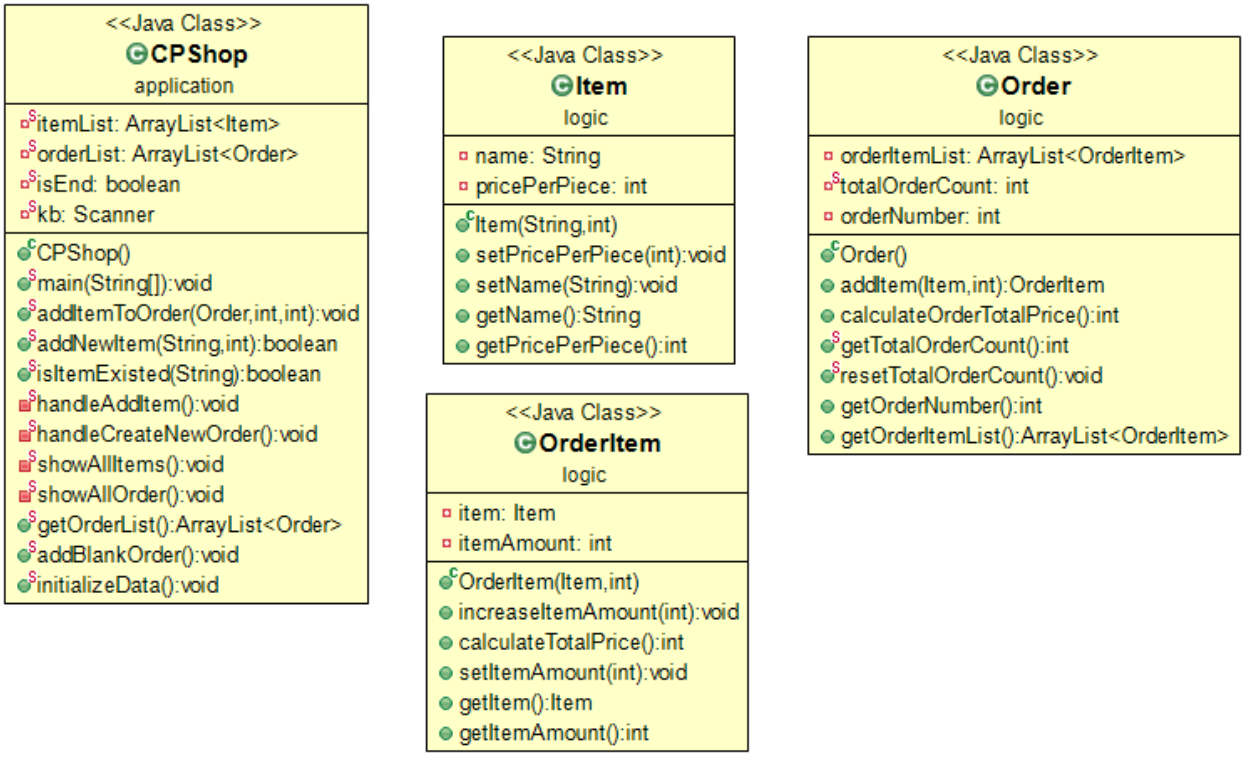

Figure 6. Class Diagram

You must write java classes using UML diagram specified above.

* In the following class description, only details of IMPORTANT fields and methods are given. *

4.1 Package logic

4.1.1 Class Item: This class represents the item.

You must implement this class from scratch.

Field

Name	Description
- String name	The name of the item.
- int pricePerPiece	Price per piece of the item.

Constructor

Name	Description
+ Item(String name, int pricePerPiece)	Create a new Item object with specified name and price per piece. Set related fields with the given parameters; Price per piece must be equals or more than 1.

Method

Name	Description
+ void setPricePerPiece(int pricePerPiece)	Set price per piece of the item. If given price per piece is less than 1, set price per piece to 1.
+ getter/setter for other variables	

4.1.2 Class OrderItem: This class is used for storing ordered Item and amount of the ordered item in the Order made through the application menu.

You must implement this class from scratch.

Field

Name	Description
- Item item	Ordered item of this OrderItem.
- int itemAmount	Amount of ordered item

Constructor

Name	Description
+ OrderItem(Item item, int itemAmount)	Create a new OrderItem object with specified item and itemAmount. Set related fields with the given parameters.

Method

Name	Description
+ void increaseItemAmount(int amount)	Increase the itemAmount by given amount. If given amount is negative values, the itemAmount should not be changed.
+ int calculateTotalPrice()	Calculate the total price of this OrderItem and return it.
+ void setItemAmount(int itemAmount)	Set itemAmount of the OrderItem by given itemAmount. If given itemAmount is negative values, set itemAmount to 0.
+ getter for each variable	

4.1.3 Class Order: This class represents a single order that is made from the application menu. An order can contain more than one item (each item has the amount to order). It is partially provided. Please fill necessary code.

/* PARTIALLY PROVIDED */

Field

Name	Description
- ArrayList<OrderItem> orderItemList	ArrayList of OrderItem.
- int totalOrderCount	Total number of orders created in the application so far. This will be used to assign order number to a new Order.
- int orderNumber	Number of this order.

Constructor

Name	Description
+ Order()	/* FILL CODE */ Create a new order object. Initialize variables, assign the orderNumber to totalOrderCount and then increase the totalOrderCount by 1.

Method

Name	Description
+ OrderItem addItem(Item item, int amount)	/* FILL CODE */ Adding item to this order by creating new OrderItem with given amount. If an OrderItem with the same item already existed, increase the ItemAmount in the OrderItem by a given amount instead.
+ int calculateOrderTotalPrice()	/* FILL CODE */ Calculate total price of the order by summing total price of each orderItem in orderItemList
+ void resetTotalOrderCount()	Reset totalOrderCount to 0.

+ int getTotalOrderCount()	Return totalOrderCount
+ other remaining getters	

4.2 Package application

4.2.1 Class CPShop: This class represents the shop. It also contains main method. This class is partially provided. You only need to fill the code where necessary.

/* PARTIALLY PROVIDED */

Field

Name	Description
- ArrayList<Item> itemList	ArrayList of item containing all items
- ArrayList<Order> orderList	ArrayList of order containing all orders

Method

Name	Description
+ void main()	Handle CPShop process.
+ addItemToOrder(Order order, int itemIndex, int amount)	/* FILL CODE */ Add item at given itemIndex from itemList to the order with given amount.

Part 2: Inheritance

1. Objective

1) Be able to understand and utilize a concept of inheritance and polymorphism in Object-Oriented Programming (OOP).

2. Instruction

1) Create Java Project named **"2110215_Midterm_Part2"**.
2) Copy all folders in **"toStudent/Part2"** to your project directory src folder.
3) You are to implement the following classes (detail for each class is given in section 3 and 4)

   a) **NormalRock** (package logic.rocks)
   b) **PoisonRock** (package logic.rocks)
   c) **GameManager** (package logic.rocks)

4) **JUnit for testing is in package test.grader**

3. Problem Statement: Terry VS. Zombies

You are developing a game called Terry VS. Zombies, a fangame based on Terraria and Plants VS. Zombies. You play an apocalyptic survivor named Terry who can pull rocks from his pouch of infinite rocks and throw them at the zombies. The rocks have different properties, such as being able to pierce through armor or being able to deal poison damage.

4. Implementation Detail

The class package is summarized below.

*** In the following class description, only details of IMPORTANT fields and methods are given. ***

4.1 Package logic.rocks

4.1.1 Class NormalRock **/* You must implement this class from scratch */**

Field

Name	Description
# int damage	How much damage this rock will deal.

Constructor

Name	Description
+ NormalRock(int damage)	Constructor method. Set the rock's damage here.

Method

Name	Description
+ void setDamage(int damage)	Set the damage. If damage is less than 0, it is set to 0.
+ int getDamage()	Returns the damage.
+ int dealDamage(Zombie zombie)	If the zombie's defense is greater than or equal to the rock's damage, decrease 0 health from the given zombie. Otherwise, get the zombie's defense, subtract it from this rock's damage, and decrease that much health from the given zombie. Finally, return the damage dealt after accounting for the zombie's defense.
+ String toString()	Prints the object as a string. Use this format: `"Normal Rock (" + getDamage() + ")"`

4.1.2 Class PoisonRock /* You must implement this class from scratch */

Note that this class is a type of rock.

Additional Fields

Name	Description
- int damageOverTime	How much decay will be added to the targeted zombie when this rock hits. (Decay will deal damage (ignoring defense) whenever a zombie takes its turn. If a zombie dies by decay, it will not take an action.)

Constructor

Name	Description
+ PoisonRock(int damage, int, damageOverTime)	Constructor method. Set the rock's damage and damageOverTime here.

Method

Name	Description
+ void setDamageOverTime(int)	Set the damageOverTime. If damageOverTime is less than 0, set it to 0.
+ int getDamageOverTime()	Returns the damageOverTime.
+ int dealDamage(Zombie)	Adds this rock's damageOverTime to the zombie's decay, then deal the damage and return the damage value like in NormalRock. **(Note: Do not use damageOverTime to reduce the zombie's health here.)**
+ String toString()	Prints the object as a string. Use this format: `"Poison Rock (" + getDamage() +", DoT = " +getDamageOverTime() +")"`

4.1.3 Class RockManager

This class is given. It creates random rock used in the game.

Method

Name	Description
+ NormalRock randomRock()	Returns a random rock with random parameters. The rock returned here can either be a NormalRock, a PierceRock, or a PoisonRock.

4.2. Package logic.game

4.2.1. Class GameManager /* PARTIALLY PROVIDED */

Additional Fields

Name	Description
+ Player player	The player object.
+ ArrayList<Zombie> zombies	The list containing the zombies the player is currently fighting.
+ NormalRock currentRock	The rock that will be used to attack when attackZombie(Zombie zombie) is called.
+ NormalRock nextRock	The rock that will replace currentRock once currentRock is used.

Method

Name	Description
+ int attackZombie (Zombie zombie)	/* FILL CODE */ Attacks the given zombie with the current rock.

4.3. Package logic. zombies (Do not modify any class in this package.)

4.3.1. Class Zombie

Fields

Name	Description
- int health	The zombie's current health. If it reaches 0, the zombie is dead and cannot take action.

- int maxHealth	The zombie's max health.
- int power	The zombie's power. When the zombie takes action, the zombie will deal this much damage to the player.
- int defense	The zombie's defense. When attacked, the zombie takes this much less damage unless attacked by a PierceRock, which ignores this defense.
- int decay	The zombie's decay. When the zombie takes action, the zombie will lose this much health (ignoring defense). If the zombie's health reaches 0 this way, they will not take action.

Method

Name	Description
Public setters for all above	Note that if a value less than 0 is given, the value is set as 0.
Public getters for all variables above	